ARTI 502

Deep Learning: 2nd Assignment

Training-Testing CNN with CIFAR Dataset using PyTorch

Year 2026 Term 1

Group # 2

Group Members:

#	Member Name	Member Academic ID
1	Mohammed Jaarran	2230000129
2	Mohaiya Aldossary	2230004579
3	Sameer Almadani	2230001818
4	Omar Bodai	2240040094
5	Hussain Alabdurabalnabi	2230004638

Task 1: Getting the CIFAR Dataset

We used torchvision.datasets.CIFAR10 with download=True to download the dataset into ./data_cifar. ToTensor changes the images into tensors and scales the pixel values to be between 0 and 1. The dataset has 50,000 training images, 10,000 test images and 10 classes.

```
###Task-1
import torch
import torchvision
import torchvision.transforms as transforms

DATA_PATH = './data_cifar'   # must be an empty directory

transform = transforms.ToTensor()

trainset = torchvision.datasets.CIFAR10(root=DATA_PATH, train=True,
                                        download=True, transform=transform)
testset = torchvision.datasets.CIFAR10(root=DATA_PATH, train=False,
                                       download=True, transform=transform)

print("Train samples:", len(trainset))   # 50000
print("Test samples:", len(testset))     # 10000
print("Classes:", trainset.classes)

Recorded output
Train samples: 50000
Test samples: 10000
Classes: ['airplane', 'automobile', 'bird', 'cat', 'deer',
          'dog', 'frog', 'horse', 'ship', 'truck']
```

Dataset loading code and recorded sample counts

Task 2: Defining the Network

Our first CNN has three convolutional layers with 32, 64 and 128 filters. Each layer uses a 3 x 3 kernel, padding of 1 and ReLU. Max pooling halves the image height and width after every convolution. The final feature map is 128 x 4 x 4, which gives 2,048 features for the fully connected classifier.

```
### Task-2.1 (Network)
import torch.nn as nn
import torch.nn.functional as F

class CNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 32, kernel_size=3, padding=1)    # 3x32x32  -> 32x32x32
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)   # 16x16    -> 64x16x16
        self.conv3 = nn.Conv2d(64, 128, kernel_size=3, padding=1)  # 8x8      -> 128x8x8
        self.pool = nn.MaxPool2d(2, 2)                             # halves H and W
        self.fc1 = nn.Linear(128 * 4 * 4, 256)
        self.fc2 = nn.Linear(256, 10)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))   # -> 32x16x16
        x = self.pool(F.relu(self.conv2(x)))   # -> 64x8x8
        x = self.pool(F.relu(self.conv3(x)))   # -> 128x4x4
        x = x.view(x.size(0), -1)              # -> 2048
        x = F.relu(self.fc1(x))                # -> 256
        return self.fc2(x)                     # -> 10 logits
```

Source code of the baseline CNN

Task 2: Training and Test Functions

The train function puts the network in training mode, clears the previous gradients, calculates the loss, runs backward and updates the weights. We use CrossEntropyLoss because the network returns ten class scores for every image. We multiply each batch loss by the batch size before calculating the average for the whole dataset.

```python
### Task-2.2 (Train and Test functions)
def train(model, loader, optimizer, criterion):
    model.train()
    total_loss = 0
    for images, labels in loader:
        images, labels = images.to(device), labels.to(device)
        optimizer.zero_grad()
        loss = criterion(model(images), labels)
        loss.backward()
        optimizer.step()
        total_loss += loss.item() * images.size(0)
    return total_loss / len(loader.dataset)

def test(model, loader, criterion):
    model.eval()
    total_loss, correct = 0, 0
    with torch.no_grad():
        for images, labels in loader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            total_loss += criterion(outputs, labels).item() * images.size(0)
            correct += (outputs.argmax(1) == labels).sum().item()
    return total_loss / len(loader.dataset), 100 * correct / len(loader.dataset)
```

Training and test function source code

The test function uses model.eval() and torch.no_grad(), so it does not update weights or calculate gradients. outputs.argmax(1) selects the class with the highest score. Accuracy is the number of correct predictions divided by the number of test images, multiplied by 100.

The functions return the average loss and test accuracy so we can record the results and draw the required graphs. For the improved model, evaluation mode also turns off dropout and uses the stored batch-normalization statistics.

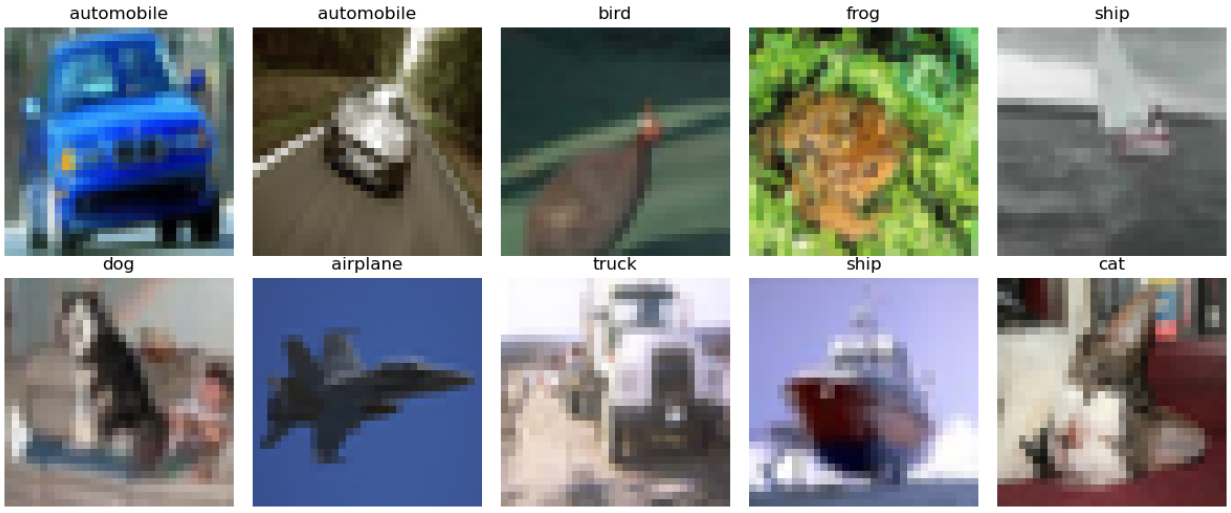
Task 3: Loading Data

We made DataLoaders with batch size 64 for the original experiments. The training loader is shuffled and the test loader is not. Then we took one mini-batch, printed its size and displayed the first ten images with matplotlib.

```
import matplotlib.pyplot as plt
import numpy as np
from torch.utils.data import DataLoader
BATCH_SIZE = 64
trainloader = DataLoader(trainset, batch_size=BATCH_SIZE, shuffle=True)
testloader = DataLoader(testset, batch_size=BATCH_SIZE, shuffle=False)
images, labels = next(iter(trainloader))
print("Images tensor size:", images.shape)   # torch.Size([64, 3, 32, 32])
print("Labels tensor size:", labels.shape)   # torch.Size([64])
print("Pixel range:", images.min().item(), "to", images.max().item())
classes = trainset.classes
fig, axes = plt.subplots(2, 5, figsize=(12, 5))
for i, ax in enumerate(axes.flat):
    ax.imshow(np.transpose(images[i].numpy(), (1, 2, 0)))   # CxHxW -> HxWxC
    ax.set_title(classes[labels[i]])
    ax.axis("off")
plt.tight_layout()
plt.show()

Recorded output
Images tensor size: torch.Size([64, 3, 32, 32])
Labels tensor size: torch.Size([64])
Pixel range: 0.0 to 1.0
```

Images tensor size and visualization code



Ten CIFAR-10 images with their class labels

1. Images tensor size

The images tensor is [64, 3, 32, 32], which means 64 images per batch, 3 color channels and 32 x 32 pixels. The labels tensor is [64], with one class label for each image.

2. Data images visualization using matplotlib

We used a 2 x 5 grid and put the class name above each image. The code changes the image order from C x H x W to H x W x C so matplotlib can display the color channels correctly.

Task 4: Dataset Augmentation

We tried all eight transform types suggested in the assignment. The training pipeline uses ToPILImage, horizontal flips, resized crops, color jitter, rotation, normalization, Gaussian noise and ToTensor. ToTensor is also used before ToPILImage so that it receives a tensor input.

```python
### Task-4
import torch
import torchvision.transforms as transforms

MEAN = (0.4914, 0.4822, 0.4465)   # CIFAR-10 channel means
STD  = (0.2470, 0.2435, 0.2616)   # CIFAR-10 channel stds

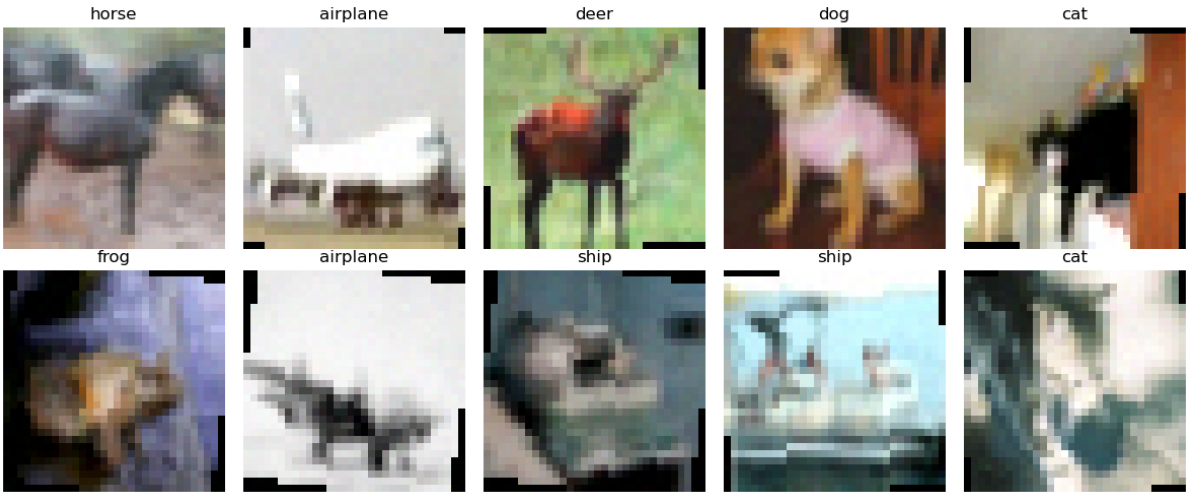
class AddGaussianNoise:
    def __init__(self, std=0.03):
        self.std = std
    def __call__(self, tensor):
        return tensor + torch.randn_like(tensor) * self.std

train_transform = transforms.Compose([
    transforms.ToTensor(),                                  # PIL -> tensor (needed so
ToPILImage gets a valid input)
    transforms.ToPILImage(),                                # 1
    transforms.RandomHorizontalFlip(p=0.5),                 # 2
    transforms.RandomResizedCrop(32, scale=(0.8, 1.0)),     # 3
    transforms.ColorJitter(brightness=0.2, contrast=0.2,
                           saturation=0.2, hue=0.05),       # 4
    transforms.RandomRotation(10),                          # 5
    transforms.ToTensor(),                                  # 8
    transforms.Normalize(MEAN, STD),                        # 6
    AddGaussianNoise(0.03),                                 # 7
])

test_transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize(MEAN, STD),
])
```

Source code of the training and test augmentation pipelines

The test data only uses ToTensor and the same normalization. We do not add random changes or noise to test images. After defining the transforms, we reload the datasets and rebuild the DataLoaders.



Augmented training images after reversing normalization for display

Task 5: Training Function and Tuning Hyperparameters

We wrote fit to train a new model and record the training loss, test loss and test accuracy after every epoch. The first three experiments use Adam, CrossEntropyLoss, batch size 64 and 10 epochs. Weight decay is 0.0 in these runs. plot_history draws the loss and accuracy curves.

```
### Task-5.1 Exp 1: baseline
import time
import torch.optim as optim

def fit(model, epochs, lr, weight_decay=0.0, scheduler_step=None):
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=lr, weight_decay=weight_decay)
    scheduler = (optim.lr_scheduler.StepLR(optimizer, step_size=scheduler_step, gamma=0.5)
                 if scheduler_step else None)
    hist = {"train_loss": [], "test_loss": [], "test_acc": []}
    for epoch in range(1, epochs + 1):
        t0 = time.time()
        tr_loss = train(model, trainloader, optimizer, criterion)
        te_loss, te_acc = test(model, testloader, criterion)
        if scheduler:
            scheduler.step()
        hist["train_loss"].append(tr_loss)
        hist["test_loss"].append(te_loss)
        hist["test_acc"].append(te_acc)
        print(f"Epoch {epoch:2d}/{epochs} | train loss {tr_loss:.4f} | "
              f"test loss {te_loss:.4f} | test acc {te_acc:.2f}% | {time.time()-t0:.0f}s")
    return hist

def plot_history(hist, title=""):
    epochs = range(1, len(hist["train_loss"]) + 1)
    fig, ax = plt.subplots(1, 2, figsize=(12, 4))
    ax[0].plot(epochs, hist["train_loss"], label="Train loss")
    ax[0].plot(epochs, hist["test_loss"], label="Test loss")
    ax[0].set_xlabel("Epoch"); ax[0].set_ylabel("Loss"); ax[0].legend(); ax[0].set_title("Loss
" + title)
    ax[1].plot(epochs, hist["test_acc"], color="green")
    ax[1].set_xlabel("Epoch"); ax[1].set_ylabel("Accuracy (%)"); ax[1].set_title("Test
accuracy " + title)
    plt.tight_layout(); plt.show()
```

Training loop and code for the required loss and accuracy plots

Experiment 1 trains the baseline CNN with learning rate 0.001. All three original runs use the augmentation pipeline from Task 4, including the run called baseline.

Changing the network and learning rate

For Experiment 2, we added batch normalization after every convolution and dropout with probability 0.3 after the 256-unit hidden layer. We kept the learning rate at 0.001 so we could compare it with the baseline.

```
### Task-5.2 Exp 2: BatchNorm + Dropout
class CNN2(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 32, 3, padding=1);   self.bn1 = nn.BatchNorm2d(32)
        self.conv2 = nn.Conv2d(32, 64, 3, padding=1);  self.bn2 = nn.BatchNorm2d(64)
        self.conv3 = nn.Conv2d(64, 128, 3, padding=1); self.bn3 = nn.BatchNorm2d(128)
        self.pool = nn.MaxPool2d(2, 2)
        self.drop = nn.Dropout(0.3)
        self.fc1 = nn.Linear(128 * 4 * 4, 256)
        self.fc2 = nn.Linear(256, 10)

    def forward(self, x):
        x = self.pool(F.relu(self.bn1(self.conv1(x))))
        x = self.pool(F.relu(self.bn2(self.conv2(x))))
        x = self.pool(F.relu(self.bn3(self.conv3(x))))
        x = x.view(x.size(0), -1)
        x = self.drop(F.relu(self.fc1(x)))
        return self.fc2(x)
```

CNN2 with batch normalization and dropout

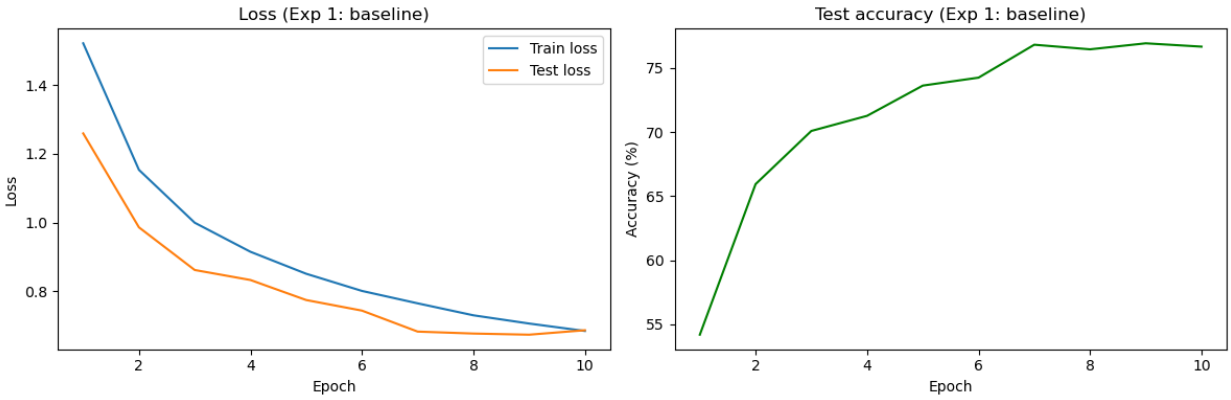
For Experiment 3, we used CNN2 again, started the learning rate at 0.002 and added StepLR. It halves the learning rate after every three epochs. Both models are initialized from new weights.

```
### Task-5.3 Exp 3: BatchNorm + Dropout + StepLR
model3 = CNN2().to(device)
hist3 = fit(model3, epochs=10, lr=2e-3, scheduler_step=3)
plot_history(hist3, "(Exp 3: BN + Dropout, lr 2e-3 with StepLR)")
print(f"Final -> train loss {hist3['train_loss'][-1]:.4f}, "
      f"test loss {hist3['test_loss'][-1]:.4f}, test acc {hist3['test_acc'][-1]:.2f}%")
```

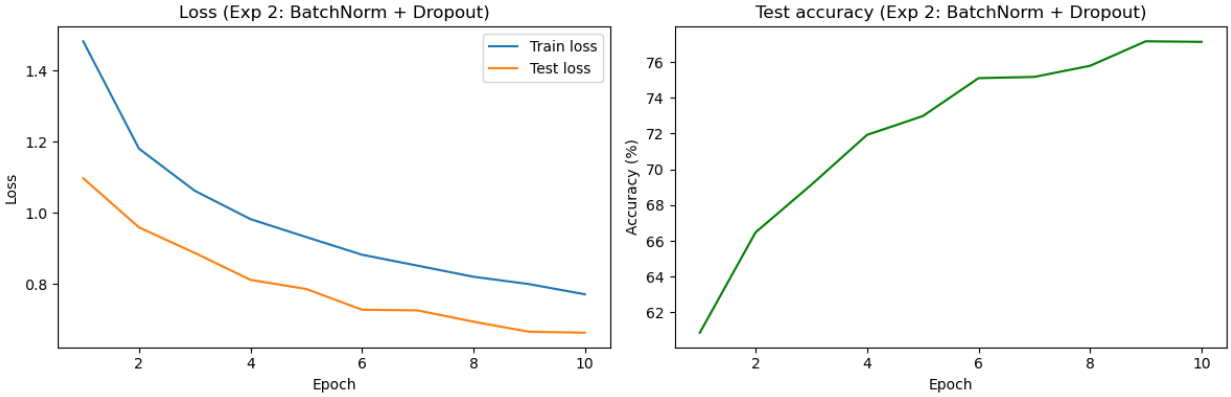
Training Experiment 3 with the stepped learning rate

Exp.	Design	Final train loss	Final test loss	Final accuracy
1	Baseline CNN	0.6843	0.6864	76.66%
2	CNN2; lr 0.001	0.7704	0.6624	77.13%
3	CNN2; lr 0.002; StepLR	0.7687	0.6501	77.31%

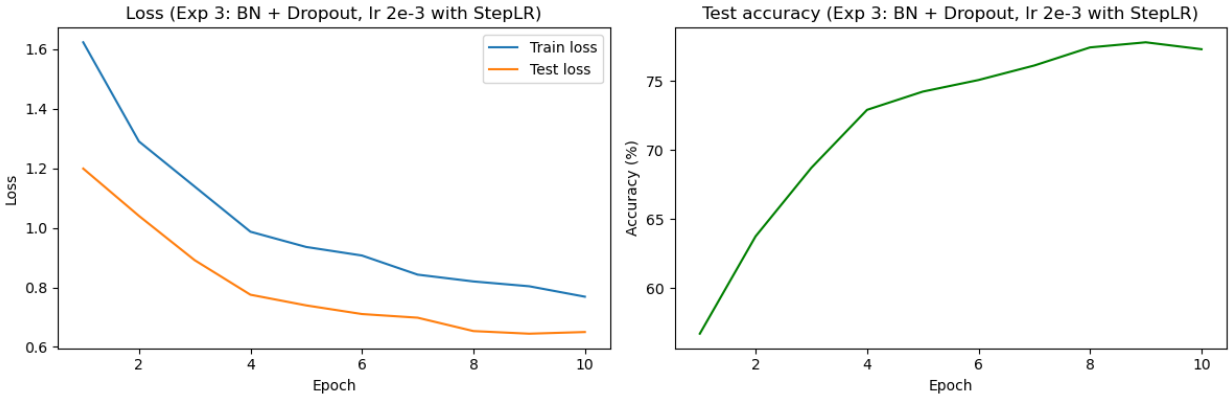
Results of the first three experiments



Experiment 1: baseline CNN



Experiment 2: batch normalization and dropout



Experiment 3: batch normalization, dropout and StepLR

Experiment 3 gave the best final result of the first three runs: train loss 0.7687, test loss 0.6501 and test accuracy 77.31%. It reached 77.81% at epoch 9 and then dropped slightly in the last epoch.

The baseline had a lower training loss, but CNN2 gave better test results. Adding the learning-rate schedule improved the final test accuracy by another 0.18 percentage points. These runs change several settings together, so they do not show the separate effect of every change.

Experiment 4: Improving the CNN

We then tried a deeper CNN with two convolutions in each block instead of one. It still uses convolution and max pooling as required. Each convolution is followed by batch normalization and ReLU. There is dropout after each pooling layer and before the final classifier. The model has 814,570 trainable parameters and is trained from scratch.

```
class ImprovedCNN(nn.Module):
    def __init__(self):
        super().__init__()
        layers = []
        in_channels = 3
        for channels in (32, 64, 128):
            for _ in range(2):
                layers += [nn.Conv2d(in_channels, channels, 3, padding=1, bias=False),
                           nn.BatchNorm2d(channels), nn.ReLU(inplace=True)]
                in_channels = channels
            layers += [nn.MaxPool2d(2), nn.Dropout2d(0.1)]
        self.features = nn.Sequential(*layers)
        self.classifier = nn.Sequential(nn.Flatten(), nn.Linear(128*4*4, 256),
                                        nn.ReLU(inplace=True), nn.Dropout(0.3),
                                        nn.Linear(256, 10))

    def forward(self, x):
        return self.classifier(self.features(x))
```

Source code of the improved CNN

For this run, we tried a lighter augmentation combination: random crops with four pixels of reflection padding, horizontal flips, scaling to [0, 1] and normalization. Each image gets its own random crop and flip. The full Task 4 pipeline remains in the code as a separate augmentation experiment.

```
def preprocess(x, augment=False):
        x = x.float().div(255)
        if augment:
            # Per-image random crop with reflection padding of four pixels.
            x = F.pad(x, (4,4,4,4), mode='reflect')
            b = x.size(0)
            offsets = torch.randint(0, 9, (b,2), device=device)
            rows = offsets[:,0,None,None] + torch.arange(32, device=device)[None,:,None]
            cols = offsets[:,1,None,None] + torch.arange(32, device=device)[None,None,:]
            x = x.permute(0,2,3,1)[torch.arange(b,device=device)[:,None,None], rows, cols]
            x = x.permute(0,3,1,2)
            flip = torch.rand(b,1,1,1,device=device) < 0.5
            x = torch.where(flip, x.flip(-1), x)
        return ((x - mean) / std).contiguous()
```

Batch preprocessing used in Experiment 4

Experiment 4: Training settings and model saving

We trained for 40 epochs with batch size 128, AdamW, initial learning rate 0.001 and weight decay 0.0005. CosineAnnealingLR gradually reduces the learning rate to 0.00001. We used seed 42 and mixed precision on an NVIDIA RTX 3060 Laptop GPU.

We split the original training set into 45,000 training images and 5,000 validation images, with the same number from every class. We saved the model with the best validation accuracy, using validation loss to break ties. We also recorded test loss and accuracy every epoch because the assignment asks for these plots. Test scores were not used to choose the saved model.

```
for epoch in range(1, epochs+1):
        tick = time.time()
        model.train()
        permutation = train_idx[torch.randperm(len(train_idx), device=device)]
        train_sum = torch.zeros((), device=device)
        correct = torch.zeros((), device=device)
        lr = optimizer.param_groups[0]['lr']
        for idx in permutation.split(batch_size):
            optimizer.zero_grad(set_to_none=True)
            with torch.autocast(device_type=device.type, enabled=device.type == 'cuda'):
                logits = model(preprocess(images[idx], augment=True))
                loss = criterion(logits, targets[idx])
            scaler.scale(loss).backward()
            scaler.step(optimizer)
            scaler.update()
            train_sum += loss.detach().float() * idx.numel()
            correct += (logits.detach().argmax(1) == targets[idx]).sum()
        val_loss, val_accuracy = evaluate(images, targets, val_idx)
        epoch_test_loss, epoch_test_accuracy = evaluate(test_images, test_labels,
test_indices)
        row = dict(epoch=epoch, train_loss=train_sum.item()/len(train_idx),
                   train_accuracy=100*correct.item()/len(train_idx),
                   validation_loss=val_loss, validation_accuracy=val_accuracy,
                   test_loss=epoch_test_loss, test_accuracy=epoch_test_accuracy,
                   learning_rate=lr, seconds=time.time()-tick)
        history.append(row)
        if val_accuracy > best_accuracy or (val_accuracy == best_accuracy and val_loss <
best_loss):
            best_accuracy, best_loss, best_epoch = val_accuracy, val_loss, epoch
            torch.save({'model_state':model.state_dict(), 'epoch':epoch,
                        'validation_accuracy':val_accuracy, 'seed':seed}, checkpoint)
        scheduler.step()
        (output/'history.json').write_text(json.dumps(history, indent=2))
        print(f"Epoch {epoch:02d}/{epochs} train loss {row['train_loss']:.4f} | "
              f"val loss {val_loss:.4f} | val accuracy {val_accuracy:.2f}% | "
              f"test accuracy {epoch_test_accuracy:.2f}% | {row['seconds']:.1f}s", flush=True)
```

Executed training loop with loss recording and validation-based model saving

Experiment 4: Full training output

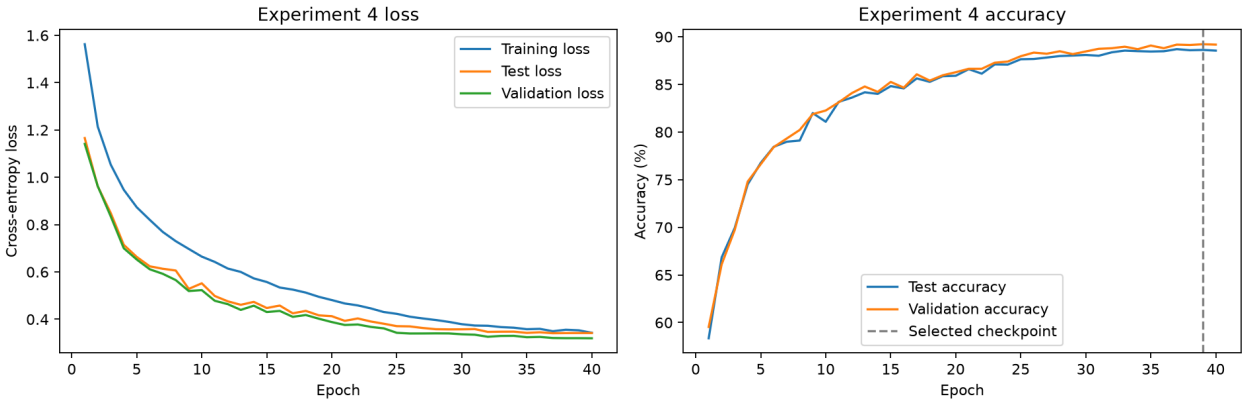
The full run completed all 40 epochs. The table below contains the measured values saved during training. Training loss decreased from 1.5625 in epoch 1 to 0.3428 in epoch 40, while final-epoch test accuracy increased from 58.35% to 88.55%.

Epoch	Train loss	Test loss	Test acc	Val loss	Val acc
1	1.5625	1.1658	58.35%	1.1411	59.54%
2	1.2142	0.9618	66.85%	0.9625	66.12%
3	1.0535	0.8490	69.93%	0.8346	69.72%
4	0.9473	0.7146	74.51%	0.7003	74.80%
5	0.8733	0.6634	76.76%	0.6524	76.62%
6	0.8202	0.6238	78.45%	0.6112	78.42%
7	0.7691	0.6134	78.98%	0.5921	79.32%
8	0.7300	0.6060	79.11%	0.5655	80.22%
9	0.6972	0.5284	81.99%	0.5192	81.88%
10	0.6648	0.5519	81.08%	0.5231	82.26%
11	0.6424	0.4984	83.17%	0.4782	83.12%
12	0.6144	0.4758	83.61%	0.4639	84.08%
13	0.6000	0.4611	84.17%	0.4396	84.78%
14	0.5727	0.4734	84.01%	0.4573	84.22%
15	0.5575	0.4476	84.82%	0.4305	85.26%
16	0.5341	0.4579	84.58%	0.4353	84.66%
17	0.5253	0.4251	85.63%	0.4102	86.06%
18	0.5126	0.4354	85.27%	0.4180	85.40%
19	0.4949	0.4166	85.86%	0.4023	85.96%
20	0.4811	0.4130	85.91%	0.3881	86.28%
21	0.4667	0.3932	86.60%	0.3758	86.64%
22	0.4585	0.4034	86.13%	0.3778	86.64%
23	0.4459	0.3906	87.10%	0.3677	87.28%
24	0.4305	0.3811	87.08%	0.3614	87.40%
25	0.4229	0.3708	87.64%	0.3427	87.96%
26	0.4109	0.3698	87.67%	0.3399	88.34%
27	0.4034	0.3631	87.82%	0.3400	88.22%
28	0.3968	0.3580	87.98%	0.3404	88.48%
29	0.3889	0.3573	88.03%	0.3401	88.18%
30	0.3791	0.3575	88.09%	0.3365	88.46%
31	0.3735	0.3585	88.01%	0.3347	88.74%
32	0.3726	0.3466	88.37%	0.3260	88.80%
33	0.3670	0.3474	88.55%	0.3294	88.96%
34	0.3643	0.3476	88.49%	0.3299	88.70%
35	0.3579	0.3424	88.45%	0.3241	89.08%
36	0.3595	0.3449	88.48%	0.3254	88.80%
37	0.3495	0.3411	88.70%	0.3207	89.18%
38	0.3557	0.3416	88.59%	0.3200	89.14%
39	0.3532	0.3418	88.62%	0.3201	89.22%
40	0.3428	0.3417	88.55%	0.3196	89.18%

Measured values from the saved 40-epoch training history

Epoch 39 had the highest validation accuracy, 89.22%, so we reloaded its saved weights and tested that model again. It classified 8,862 of the 10,000 test images correctly, giving 88.62% test accuracy and test loss 0.3418.

Experiment 4: Loss and accuracy plots



Training loss, test loss and accuracy across all 40 epochs

The graphs show that the model learns quickly at the beginning and improves more slowly near the end. Training loss can be higher than test loss because training uses random augmentation and dropout. Test images are normalized without random augmentation, and dropout is disabled during evaluation.

Metric	Selected model	Final epoch
Epoch	39	40
Training loss	0.3532	0.3428
Test loss	0.3418	0.3417
Test accuracy	88.62%	88.55%

What design worked best?

The best saved model is the deeper CNN from Experiment 4 with batch normalization, dropout, lighter augmentation, AdamW and a cosine learning-rate schedule. Its test accuracy is 88.62%, which is 11.31 percentage points higher than the best original final result of 77.31%. The training loss at the selected epoch is 0.3532 and its test loss is 0.3418.

This comparison is for the complete configuration. We changed the architecture, batch size, optimizer, augmentation and number of epochs together. Also, Experiment 4 uses 45,000 training images and a separate validation set, while the original runs use all 50,000 training images. Each configuration was run once.

Task 6: Save Your Source Code

All source code is saved in "Assignment_2_Code.ipynb" and "Assignment_2_Code.py". The notebook includes the original experiments, the improved model, the recorded training output and all plots. The saved model weights and detailed training results are also included in the code package.

Group GitHub repository: https://github.com/mohaiyaaldossary/ARTI502-Assignment-2

The source code, report, trained model and measured results are available in the Assignment 2 repository.